



University
of Windsor

SCHOOL OF COMPUTER SCIENCE

COMP2120-1-R-2025S
Object Oriented Programming using
Java
Summer 2025

Sample Final Exam

Total Marks:

Total Time: 150 Minutes

Start Time: 12.00 PM End Time: 2.30 PM

<i>Student Name</i>	
<i>Student ID</i>	

Notes:

1. You must write all your answers within the space provided in the question. No extra pages will be provided.
 2. You are not allowed to search on the internet to find any help or look at the slides.
 3. Don't spend too much time on a single problem. If you get stuck, move on to something else and come back later.
 4. Comment your code. It may help you get partial marks if the code is not correct.
-

Regular Questions	1	2	3	Total
Total Mark	20	16	14	50
Your Mark				

1. State whether the following statements are true or false (Answer: T/F)? (20 x 1 = 20 Marks)

- a. The following is an example of a valid identifier: **player***
- b. A Java program can have two variables with the following name: rate, Rate
- c. The output of the following code is 2.0:

```
double val=(1/2)*2;  
System.out.println(val)
```
- d. The trim () method in class String removes the leading whitespace only.
- e. There are three ways to add comments in Java.
- f. The unary operators (i.e., +, -, ++, --, and !) has the highest precedence in Java.
- g. The controlling expression in switch (i.e., switch (controlling_expression) {...}) can be evaluated to a String type.
- h. The following boolean expression will be evaluated to **True** if x is 9 and y is 13:
(x > 7 & y < x)
- i. The body of the do-while loop will never be executed if the condition is evaluated to **False**.
- j. It is possible to write a for loop as follows:

```
for(;;){  
    //code goes here  
}
```
- k. The main method in Java returns an integer value.
- l. When comparing two quantities of **int** type, we should use the method equals.
- m. A method definition can include call to another method that is either in the same class or in a different class.
- n. Every class in Java will contain a default constructor by default.
- o. You cannot call a static method in Java using **this** keyword.
- p. The following code is not valid considering the class DimensionConverter:

```
DimensionConverter dc = new DimensionConverter();
```

double inches = dc.convertFeetToInches(2.5);

```
/**
 * Class of static methods to perform dimension conversions.
 */
public class DimensionConverter
{
    public static final int INCHES_PER_FOOT = 12;
    public static double convertFeetToInches(double feet)
    {
        return feet * INCHES_PER_FOOT;
    }
    public static double convertInchesToFeet(double inches)
    {
        return inches / INCHES_PER_FOOT;
    }
}
```

A static constant; it could be private here.

- q. It is possible to have two classes with the same name if they are located in different packages.
- r. The following is an example of a valid code:

```
int[] b = new int[10];
for (int i = 1; i <= b.length; i++)
    b[i] = 5 * i;
```
- s. An interface can contain any number of public methods and public named constants.
- t. A constructor name in a generic class can have a type parameter (e.g., public DoublyLinkedList<E>() {...})

2. Answer the following questions (**Each contain 2 Marks**):

a) What will be the output of the following code fragment? (**2 Marks**)

```
int p = 1;
int c = 1;
for (int i = 1; i <= 5; i = i +2)
{
    p = p * i;
    for (int j = 1; j <= i; j++)
    {
        c = c * j;
    }
    p = p + c;
    System.out.println("C is " + c);
}
System.out.println("P is " + p);
```

b) Write down two differences between parameter and arguments. (**2 Marks**)

c) What is the difference between this and super when these words are used within a constructor definition as the names of methods that are called? (**2 Marks**)

d) What will be the output of the following code: **(2 Marks)**

```
int[][] testArray = new int[3][4];
for (int row = 0; row < testArray.length; row++)
    for (int col = 0; col < testArray[row].length; col++)
        testArray[row][col] = col;
for (int row = 0; row < testArray.length; row++)
{
    for (int col = 0; col < testArray[row].length; col++)
        System.out.print(testArray[row][col] + " ");
    System.out.println();
}
```

e) Write down two differences between overloading a method and overriding a method **(2 Marks)**

f) What will be the output of the following code:

(2 Marks)

Suppose the exception class CrazyException is defined as follows:

```
public class CrazyException extends Exception
{
    public CrazyException()
    {
        super("Crazy exception thrown!");
        System.out.println("Wow, Crazy exception thrown!");
    }
    public CrazyException(String message)
    {
        super(message);
        System.out.println("Wow, crazy exception thrown with "+
            "an argument!");
    }
    public void crazyMethod()
    {
        System.out.println("Message is " + getMessage());
    }
}
```

What output would be produced by the following unlikely code?

```
CrazyException exceptionObject = new CrazyException();
System.out.println(exceptionObject.getMessage());
exceptionObject.crazyMethod();
```

3. Develop code based on the following instructions **(6 Marks)**:

- a) Create an abstract class DiscountPolicy. It should have a single abstract method computeDiscount that will return the discount for the purchase of a given number of a single item. The method has two parameters, count and itemCost (i.e., cost of each item). **(2 Marks)**
- b) Derive a class BulkDiscount from DiscountPolicy, as described in the previous exercise. **(1 Mark)**
- c) It should have a constructor that has two parameters, minimum and percent. **(1 Mark)**

- d) It should define the method `computeDiscount` so that if the quantity purchased of an item is more than minimum, you will provide the percent discount based on the total cost of the items. Otherwise, no discount will be provided (i.e., the method will return zero). **(2 Marks)**